Participant's Section

I. Use Case Short Title

Reg IRIS – Intelligent Regulatory Repository & Impact Scanner

II. Problem Statement & Background

Investment banks operate within a complex global regulatory landscape, required to file trade reports to multiple regulators (e.g., MiFID II, Dodd-Frank, EMIR, FCA). The rules governing "what, when, and how" to report are not centralized; they are scattered across thousands of unstructured PDFs, HT.0ML web pages, and internal policy documents. Furthermore, these regulations are dynamic and change frequently.

Currently, compliance teams rely on manual intervention, reading unstructured text, extracting specific clauses, and cross-referencing them against the bank's reporting schema. This manual process is:

1. **Slow:** Inefficient rule-lookup times.
2. **Risky:** High probability of human error or misinterpretation.
3. **Expensive:** A single reporting error can trigger multi-million dollar regulatory fines and severe reputational damage.

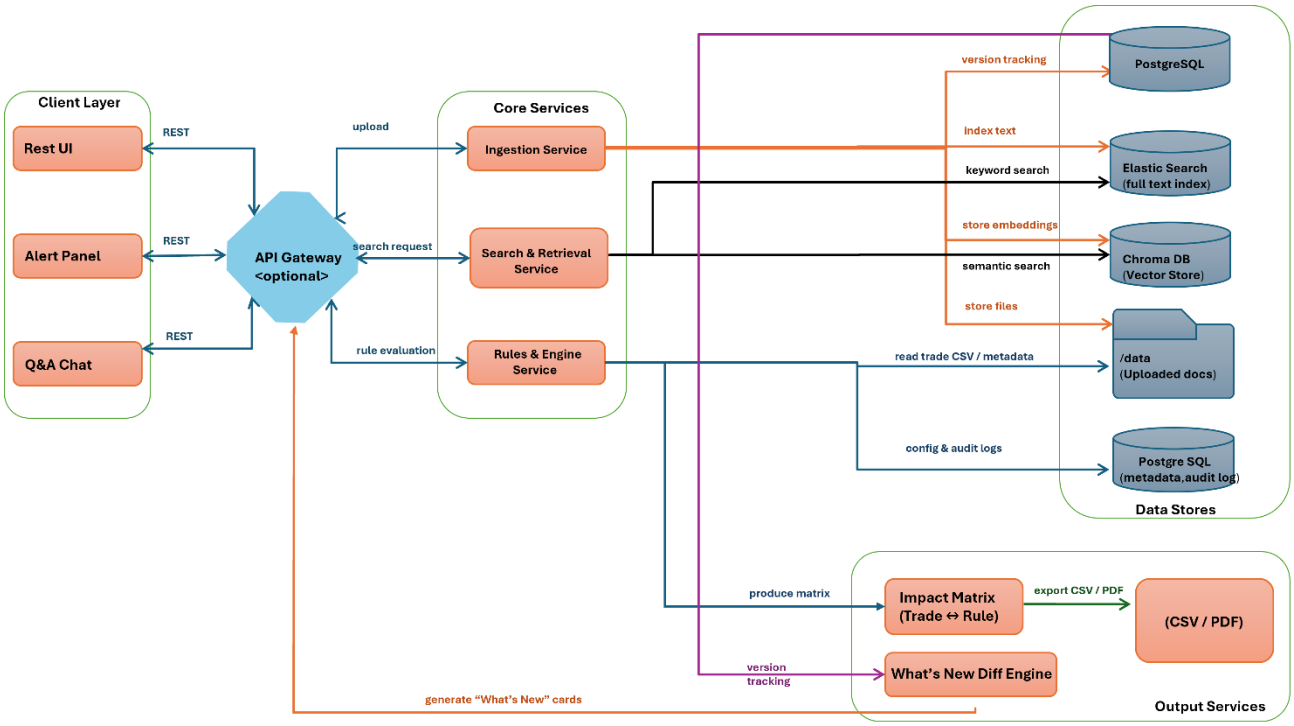
III. Objective

To build a microservices-based regulatory intelligence hub that transforms unstructured regulatory text into actionable, machine-readable compliance rules. The solution will be architected around three primary functional domains to ensure scalability and separation of concerns:

- **Regulatory Ingestion & Versioning Domain:** Managed by the **Ingestion Service**, this domain handles the end-to-end lifecycle of regulatory documents. It is responsible for file uploads, parsing unstructured text, maintaining version history (via a database-driven versioning layer), and populating the primary data stores (Elasticsearch for text indexing and ChromaDB for vector embeddings).
- **Intelligent Search & Retrieval Domain:** Managed by the **Search & Retrieval Service**, this domain provides the interface for data discovery. It enables high-precision retrieval through both keyword-based (Elasticsearch) and semantic (ChromaDB) search patterns, allowing users to find exact regulatory clauses and citations.
- **Impact Domain:** Managed by the **Rules & Engine Service** and **Output Services**, this domain transforms raw data into business intelligence. It utilizes a JSON-driven engine to perform automated impact analysis between trade datasets and regulatory mandates, generating structured outputs such as **Impact Matrices (trade-to-rule mapping)**, and automated **"What's New"** diff alerts.

Below is the high-level architecture

< Reg IRIS ><Mumbai>



IV. Definition of Done

- ✓ UI dashboard displays a "What's New" card that highlights the regulator, effective date and a short diff whenever a new document is uploaded.
- ✓ Search interface returns the exact paragraph or clause together with a clickable citation (document name + page).
- ✓ **Rule-engine** evaluates a sample trade CSV against the JSON rule set and produces an **Impact Matrix** (trade ↔ rule) with an option to export as CSV.
- ✓ All services start with a single docker-compose up command (startup ≤ 1 minute).
- ✓ REST endpoints (search, gap-analysis, impact-matrix, checklist, alerts) are documented via an OpenAPI/Swagger spec.
- ✓ A minimal suite of unit tests (≥ 8) passes in the CI pipeline.
- ✓ *(Optional)* An LLM-powered chat window answers natural-language questions and always cites the source paragraph.

V. Expected Outcomes

The prototype will deliver concrete, measurable value for the compliance function:

- **Instant regulatory-search dashboard** – users type a natural-language query and receive the exact paragraph (or clause) together with a clickable citation (document name + page). This replaces the manual "open-PDF-scroll-find" workflow.
- **"What's New" alerts** – each time a regulator publishes an update, the system creates a concise card (regulator, effective date, short diff) and pushes a mock notification to the **Alert Panel**.

- **Impact Matrix visualization** – a tabular view that instantly shows which trades (by Trade_ID) violate which rule, with a one-sentence reason (e.g., "Price field missing – required by EMIR Rule 4.2"), having an option to export to CSV.
- **(Optional) AI-assisted Q&A** – a lightweight LLM chat window answers follow-up questions and always cites the source paragraph, demonstrating how conversational assistance can be safely grounded in regulatory text.

Suggested Technology Stack (Not mandatory)

To ensure compatibility with the microservices architecture and the Docker-compose requirement, participants are encouraged to consider the following technologies:

1. Core Backend & Orchestration

- **API Framework:** FastAPI (Python) – *Highly recommended for its native OpenAPI/Swagger support and high performance.*
- **Task Processing:** Celery or BackgroundTasks (FastAPI) – *For handling heavy PDF parsing and indexing without blocking the API.*
- **Containerization:** Docker & Docker-compose – *Mandatory for the single-command startup requirement.*
- **CI/CD:** GitHub Actions – *To automate linting, testing, and image building.*

2. Data & Search Layer

- **Relational Database:** PostgreSQL or SQLite – *To manage document metadata, version history, and user logs.*
- **Full-Text Search Engine:** Elasticsearch – *For high-speed keyword-based retrieval and filtering.*
- **Vector Database (Advanced):** ChromaDB, FAISS, or Qdrant – *To store sentence embeddings for semantic search.*
- **Data Manipulation:** Pandas – *The standard for processing the Sample Trade CSV and performing rule-based validation.*

3. Frontend (Dashboard)

- **Framework:** React.js – *For a dynamic, single-page application experience.*
- **UI Component Library:** Material UI (MUI) or Tailwind CSS – *To allow for rapid development of professional, "banking-grade" dashboards.*
- **State Management:** TanStack Query (React Query) – *Recommended for efficient data fetching and synchronization with the FastAPI backend.*

4. AI & Natural Language Processing (Advanced Tier)

- **Orchestration:** LangChain or LlamaIndex – *To manage the RAG (Retrieval-Augmented Generation) pipeline.*
- **Embeddings Model:** all-MiniLM-L6-v2 (via sentence-transformers) – *A lightweight, high-performance model for local semantic search.*
- **LLM Execution:** Ollama – *The preferred method for running LLMs locally within a Docker container without requiring external API keys.*

5. Recommended LLMs (Local/Private Deployment)

Since the requirement emphasizes data privacy and local execution, participants should use small, high-reasoning models that can run on standard hardware:

- **Mistral-7B-Instruct:** *Best all-rounder for reasoning and following complex instructions.*
- **Llama-3 (8B):** *Excellent for natural language understanding and conversational Q&A.*
- **Phi-3 Mini (3.8B):** *Highly recommended if hardware resources are limited; incredibly fast and surprisingly capable for its size.*

< Reg IRIS ><Mumbai>

VI. Solution Requirements (Good to Have)

- **Semantic Intelligence:** Use vector embeddings to allow natural language querying (e.g., asking a question rather than searching keywords).
- **Zero-Code Rule Updates:** The system should allow new rules to be added via JSON configuration without requiring backend code changes.
- **Data Integrity & Grounding:** The solution must prioritize accuracy by ensuring all AI-generated insights or search results are strictly grounded in the provided source documents (to prevent LLM hallucinations).
- **Auditability & Traceability:** The solution should provide clear "lineage"—users must be able to trace a flagged trade error back to the specific regulatory clause and document version that triggered it.
- **Extensibility (Plugin-Ready):** The architecture should be designed as a microservice, allowing the Rule-Impact Engine to be exposed via REST APIs for integration into existing banking trade-flow pipelines.
- **Operational Efficiency:** The ingestion and rule-execution engine should be capable of processing regulatory updates and validating large batches of trade data with minimal latency.
- **Interpretability:** The solution must not just flag an error but provide an "explanation layer"—clearly stating the logic (e.g., "Field X is mandatory under Rule Y") to enable human oversight.

Software Engineering Excellence Checklist

Area	Expectation
Version Control	Code maintained in Git with meaningful commits.
Dependencies	Reproducible build using pinned dependency versions.
Configuration & Secrets	No hard-coded credentials or API keys.
Code Quality	Readable, modular, maintainable code.
Error Handling	Graceful handling of invalid inputs and failures.
Logging	Basic operational logging for troubleshooting.
Testing	Demonstrate positive and negative test scenarios.
Documentation	README explaining setup, execution, architecture, and assumptions.
Build & Run	Solution can be installed and executed using documented steps.
Final Validation	Team demonstrates a clean working build during evaluation.

VII. Detailed breakdown of Tasks

i. Basic Tasks – Must Have

- Build a FastAPI endpoint that accepts PDF, HTML or TXT files and stores them in a predefined /data folder.
- Parse the uploaded files to extract plain text, page numbers and basic metadata (document name, upload date).
- Index the extracted text in Elasticsearch, creating fields for full-text search and for document-level filters (regulator, effective date, category).
- Create a simple React dashboard that:
  - Shows a searchable input box.

        - Displays the returned paragraph together with the citation.
    - Implement a JSON-based rule engine that reads a predefined json rules file (sample provided in **/rules/** folder) and evaluates each rule against independent trade CSV files provided in the /trades/ folder (**Note**: The uploaded PDFs are used strictly for the Search/Retrieval engine, not for rule generation)."
    - Produce an "Impact Matrix" table that lists every trade ID, the rule it violates and a short explanatory message.
    - Wire up a relational versioning layer that records every document upload and automatically generates a "What's New" card on the dashboard.
    - Provide a single docker-compose up command that launches the frontend, backend, Elasticsearch and the relational store.

ii. Intermediate Tasks - Must Have

- Enhance the dashboard with an Alert Panel that pops up a mock notification whenever a new document is uploaded, linking directly to the "What's New" card.
- Implement filter controls on the search page (regulator, jurisdiction, effective date, category).
- Add pagination and sorting to the Impact Matrix for easier navigation of large trade sets.
- Enable one-click download of the Impact Matrix as CSV.
- Create OpenAPI documentation for all endpoints and expose a Swagger UI page.
- Write a minimal suite of unit tests covering ingestion, indexing, search, rule evaluation and API responses.
- Set up a GitHub Actions workflow that runs the linter, unit tests and builds the Docker images on every push.

iii. Advanced Tasks – Good to Have

- Generate sentence embeddings for every paragraph using a model such as all-MiniLM-L6-v2 and store them in a vector store (ChromaDB or FAISS) to enable semantic search.
- Add a switch in the UI that lets the user choose between keyword search and semantic search.
- Integrate a lightweight LLM (via LangChain) to provide a conversational Q&A assistant; each answer must cite the source paragraph(s).
- Containerize the optional vector store and LLM service so they can be started with the same docker-compose command.
- Add basic authentication (e.g., a static Bearer token) to protect the API endpoints during the demo.

VIII. Input

- **Regulatory Documentation** – collection of PDF, HTML and TXT files from regulators (SEC, FCA, ESMA, CFTC, etc.). which is openly available on the web
- **Trade-Reporting Schema** – Sample JSON or CSV file describing reporting fields (e.g., Trade_ID, Asset_Class, Timestamp, Counterparty_LEI, Price, Quantity).
- **Sample Trade File** – CSV containing mock trades that will be fed to the rule-engine for impact analysis.
- **Optional Metadata** – CSV/JSON tagging each document with regulator, jurisdiction, effective date and category (used for filtering and "What's New" diff).

- **User Queries (optional)** – plain-text file with example natural-language questions for testing the search or chatbot.

IX. Guardrails

- **No real client data** – only synthetic, anonymised files may be used.
- **No filing to actual regulators** – the prototype only detects gaps; it does not submit reports.
- **Rule definitions must be data-driven** – the engine may not contain hard-coded if/else logic; all rules are expressed in JSON.
- **No external web-scraping** – all regulator documents are provided beforehand; the solution must not depend on live internet access.

X. Dataset

- **/regulations/:** 10+ diverse regulatory PDFs and HTML files.
- **/schema/:** reporting_schema.json.
- **/trades/:** sample_trades.csv (approx. 200 rows with deliberate errors like missing LEIs, negative prices, or invalid timestamps).
- **/metadata/:** doc_metadata.csv.
- **/queries.txt/:** sample natural-language queries for testing
- **/rules/:** json file containing validation rules for different regulation regimes

XI. Reference

- **MiFID II** – European Securities and Markets Authority (ESMA) handbook (public PDFs).
- **Dodd-Frank Act** – U.S. Securities and Exchange Commission (SEC) guidance documents.
- **FastAPI Documentation** – https://fastapi.tiangolo.com/
- **React Material-UI** – https://mui.com/getting-started/installation/
- **Elasticsearch Guide** – https://www.elastic.co/guide/en/elasticsearch/reference/current/index.html
- **JSONLogic** – https://jsonlogic.com/ (rule expression format).
- **Sentence-Transformers** – https://www.sbert.net/ (for optional semantic search).
- **LangChain Retrieval-QA** – https://python.langchain.com/docs/use_cases/question_answering/ (LLM chat grounding).